Maximum Likelihood in Machine Learning: Losses and Priors

Contents

1. Overview . . . 2
2. A model as a distribution of the target . . . 3
3. Gaussian noise gives least squares . . . 5
   - 3.1 The model . . . 5
   - 3.2 From the NLL to squared errors . . . 5
   - 3.3 The same answer as least squares . . . 6
   - 3.4 Estimating the noise . . . 7
4. A Bernoulli target gives the log loss . . . 8
   - 4.1 Why least squares cannot fit logistic regression . . . 8
   - 4.2 Project, convert, multiply, turn . . . 9
   - 4.3 The Bernoulli model behind the log loss . . . 10
5. A categorical target gives the cross entropy . . . 12
   - 5.1 Why not squared residuals? . . . 15
6. Maximum likelihood overfits . . . 16
7. MAP estimation: maximum likelihood plus a prior . . . 18
   - 7.1 The posterior over the parameters . . . 18
   - 7.2 A Gaussian prior gives ridge regression . . . 19
   - 7.3 A Laplace prior gives the lasso . . . 21
8. Summary . . . 23
9. Sources . . . 24
10. Key terms . . . 24

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Overfitting; One-hot encoding; Bayes' theorem; Maximum likelihood estimation (MLE).
- **Used here, taught in full later:** Softmax regression.
- **Leads to:** Ridge regression; Lasso regression; Logistic regression; Gradient boosting; Cost-sensitive learning; ANN for classification; Loss functions in deep learning.
- **Compare with:** Hinge loss and soft margin; Entropy, information gain and Gini.

1. Overview

Key point: The usual ML losses are negative log-likelihoods. Gaussian noise gives the mean squared error, a Bernoulli target gives the log loss, a categorical target gives the cross entropy. Adding a **prior** (G-1564) on the parameters (MAP) gives regularisation.

This Note follows *Mathematics for Machine Learning* (Deisenroth, Faisal and Ong, 2020; MML below), Section 8.3 (Parameter Estimation) and Section 9.2 (Parameter Estimation for linear regression).

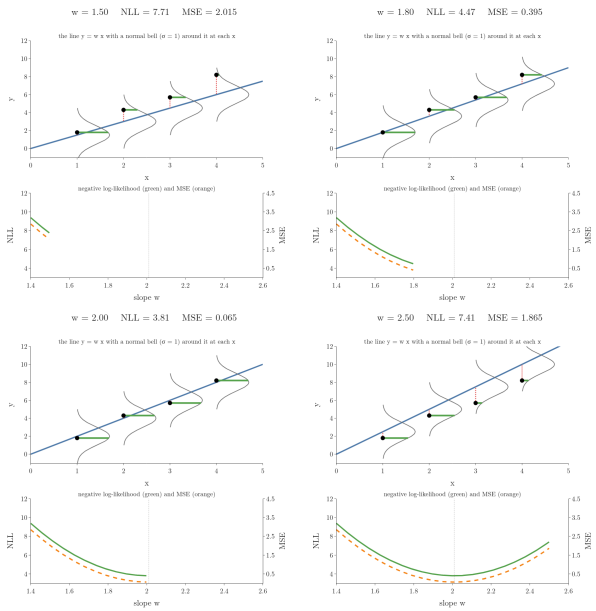


Figure 1: A line y = wx through four points, with the slope sweeping from 1.4 to 2.6. Each grey bell is the normal distribution of y around the line; the green bar is its height at the observed y. Bottom: the negative log-likelihood and the mean squared error against w, lowest at the same slope

Maximum likelihood estimation (G-1191) fitted one distribution to a column of numbers.

In ML we predict a **target** y (the output we want) from **features** x (the input variables, one column each of the data table). One record, the features of one case together with its target, is an **observation** (one row of the data table). The distribution of y changes with x. Figure 1 shows the idea for regression: around the line sits a bell, and a good line puts every point near the top of its bell.

Figure 2 is the map of this Note: read it from left to right, from a model of the target, through maximum likelihood, to a loss (or from a prior, through MAP, to a penalty). Each model of the target turns, through maximum likelihood, into a loss we already know; each prior on the parameters turns, through MAP estimation, into a penalty we already know.

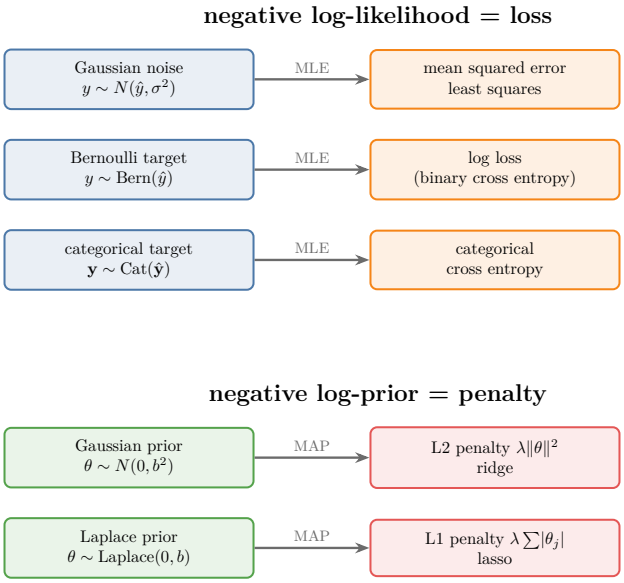


Figure 2: Models of the target become losses under maximum likelihood; priors on the parameters become penalties under MAP

2. A model as a distribution of the target

> **Key point:** A probabilistic model does not say "the answer is 5"; it says how likely each possible answer is. The score of a model on the training set is how likely it found the answers that really happened.

In plain words. A weather app says "70 percent chance of rain", not just "rain". That is a model that gives a probability to each possible outcome. Training should make the model give high probability to what actually happened. A model that said 90 percent for the real outcomes beats one that said 10 percent.

Worked example. Three training observations (rows of the data table). For each, the model gave a probability to the target that was truly observed:

Observation	Probability given to the true target	Minus its natural log
1	0.9	$-\log 0.9 = 0.105$
2	0.5	$-\log 0.5 = 0.693$
3	0.8	$-\log 0.8 = 0.223$
Total		$0.105 + 0.693 + 0.223 = 1.02$

A probability near 1 costs almost nothing, a probability near 0 costs a lot, and the total is the **negative log-likelihood** (NLL). Smaller is better. The joint probability of the three outcomes is their product (the observations are independent):

$$0.9 \times 0.5 \times 0.8 = 0.36$$

Its negative log is the same total:

$$-\log 0.36 = 1.02$$

Taking logs turns the product into a sum.

The formal version. A model with parameters θ (all the weights, for example $\theta = (w, b) = (2, 0.5)$ for a line) takes the features x (for example $x = 3$ hours of study) and returns a distribution over the possible targets y. Such a model is a **probabilistic model** (G-1567). We write it as a conditional distribution (see conditional probability):

$$p(y \mid x, \theta)$$

Read it as "the probability of the target y, given the features x and the parameters θ". In the table above, $p(y_1 \mid x_1, \theta) = 0.9$. For the training observations $(x_1, y_1), \ldots, (x_n, y_n)$, here $n = 3$, we assume they are independent and identically distributed (i.i.d.), so the likelihood is a product over observations and the NLL is a sum. The symbol $\arg\min_\theta$ means "the θ that gives the smallest value":

$$\mathrm{NLL}(\theta) = -\sum_{i=1}^{n} \log p(y_i \mid x_i, \theta)$$

$$\hat{\theta}_{\mathrm{ML}} = \arg\min_{\theta} \mathrm{NLL}(\theta)$$

Check: with $n = 3$ the sum has three terms, as in the table:

$$-(\log 0.9 + \log 0.5 + \log 0.8) = 1.02$$

Two choices define a model: how the prediction depends on x (a line, a sigmoid, a neural network) and which distribution describes the target around that prediction. The second choice decides the loss.

> **Extra:** MML (§8.3.1) warns that θ standing to the right of the bar in $p(y \mid x, \theta)$ does not make it fixed. In the NLL, the data is fixed and θ is the variable, exactly as in likelihood is a height.

3. Gaussian noise gives least squares

> **Key point:** If y is the prediction plus normal **noise** (G-1326), the NLL is the sum of squared errors divided by $2\sigma^2$, plus a constant. Minimising it is least squares.

First, the usual way to fit a line, from the error function. Measure each point's **residual** (G-705), the vertical gap between the point and the line; square it, so that gaps above and below the line cannot cancel; add the squares up. The total is the **sum of squared errors** (G-1684). Turn the line a little and measure again. The line with the smallest total is the **ordinary least squares** (G-1406) line. This section shows that maximum likelihood, with one assumption about the noise, picks exactly the same line.

3.1 The model

> **Key point:** $y = \hat{y} + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$: the target is normal around the prediction.

Linear regression predicts $\hat{y} = wx + b$ (see the best-fit line). The points never lie exactly on the line. We model the gap as **noise** (G-1326): a random error ε drawn from a normal distribution with mean 0 and standard deviation σ (see parameters and notation). Noise of this kind is called **Gaussian noise** (G-831). Then

$$p(y \mid x, \theta) = N(y \mid \hat{y}, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}}\, e^{-\frac{(y-\hat{y})^2}{2\sigma^2}}$$

In Figure 1 each grey bell is this distribution at one x. Its peak sits on the line, and its height at the observed y is that observation's likelihood (green bar). The bell model is the "normality of **residuals** (G-705)" assumption of assumption 3, normal residuals, now used to derive the loss.

3.2 From the NLL to squared errors

> **Key point:** The log of a normal density is a constant minus $(y - \hat{y})^2/(2\sigma^2)$. So the NLL is the squared error plus a constant.

1. **In words:** the log of the normal density splits into a part that does not depend on the weights and minus the squared error over $2\sigma^2$ (as in the log-likelihood one step at a time). With σ fixed, only the squared errors change with the weights.

2. **Formula:**

$$\mathrm{NLL}(\theta) = \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + n\log\left(\sigma\sqrt{2\pi}\right)$$

   The second term is a constant, and the factor $1/(2\sigma^2)$ only rescales the first. So the θ with the smallest NLL is the θ with the smallest **sum of squared errors** (G-1684), and also the smallest mean squared error.

3. **Example:** four points $x = 1, 2, 3, 4$ with $y = 1.8, 4.3, 5.7, 8.2$, the line $\hat{y} = wx$ and $\sigma = 1$. At $w = 2$ the predictions are $2, 4, 6, 8$, so the errors $y - \hat{y}$ are $-0.2, 0.3, -0.3, 0.2$. One step per line:

$$\text{squared sum} = 0.04 + 0.09 + 0.09 + 0.04$$

$$\text{squared sum} = 0.26$$

$$n\log\left(\sigma\sqrt{2\pi}\right) = 4 \times 0.919 = 3.676$$

$$\mathrm{NLL}(2) = \frac{0.26}{2} + 3.676$$

$$\mathrm{NLL}(2) = 0.13 + 3.676 = 3.81$$

$$\mathrm{MSE}(2) = \frac{0.26}{4} = 0.065$$

   At $w = 2.5$: NLL 7.41 and MSE 1.865. Both are worse.

The bottom panel of Figure 1 draws both curves. They have different heights, but their lowest points are at the same slope.

3.3 The same answer as least squares

> **Key point:** Setting the derivative to 0 gives the **ordinary least squares** (G-1406) formula; for many features, the normal equation.

For the line through the origin, the derivative of $\sum(y_i - wx_i)^2$ with respect to w is $-2\sum x_i(y_i - wx_i)$. Setting it to 0:

$$\hat{w} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

$$\hat{w} = \frac{1.8 + 8.6 + 17.1 + 32.8}{1 + 4 + 9 + 16}$$

$$\hat{w} = \frac{60.3}{30} = 2.01$$

With an intercept and many features, the same derivation gives the normal equation $\hat{\beta} = (X^\top X)^{-1} X^\top y$ (the normal equation). So ordinary least squares, the mean squared error and maximum likelihood with **Gaussian noise** (G-831) all give the same line.

The NLL here is a quadratic bowl in the weights. MML (§9.2.1, remark after equation 9.12) shows that its Hessian, $X^\top X$, is positive definite, so the single flat point is the global minimum (convexity: a straight line gives one bowl).

3.4 Estimating the noise

> **Key point:** Maximum likelihood also estimates σ^2: it is the mean squared **residual** (G-705) of the fitted line.

Treating σ as a parameter too and setting its derivative to 0 (MML §9.2.1, equation 9.22) gives the same result as the normal MLE variance of the MLE of the standard deviation, with the residuals in place of the distances from the mean:

1. **In words:** the MLE of the **noise** (G-1326) variance is the average squared distance between the targets and the fitted line.
2. **Formula:**
$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$
3. **Example:** at $\hat{w} = 2.01$ the squared residuals add up to 0.257, so:

$$\hat{\sigma}^2 = 0.257/4 = 0.064$$

$$\hat{\sigma} = \sqrt{0.064} = 0.25$$

   The points scatter about 0.25 above and below the line.

So the training MSE of a least squares fit is the maximum likelihood estimate of the noise variance.

> **Extra:** Choosing a different noise distribution gives a different loss. With **Laplace noise**, drawn from the **Laplace distribution** (G-1044) with density $e^{-|y-\hat{y}|/b}/(2b)$, the log of one density is $-\log(2b) - |y - \hat{y}|/b$. Summing and changing the sign, the NLL is $\sum_i |y_i - \hat{y}_i|/b + n \log(2b)$: with b fixed, minimising it minimises the absolute errors, the mean absolute error (MAE), called the L1 loss in mean absolute error as a loss. The

Notebook checks this by minimising the Laplace NLL numerically. The Laplace curve has heavier tails than the normal curve, so a far-away point is less surprising under it; in loss terms, an outlier adds its distance, not its squared distance. The heavier tails are why the absolute-error loss is more robust to outliers (Murphy §7.4).

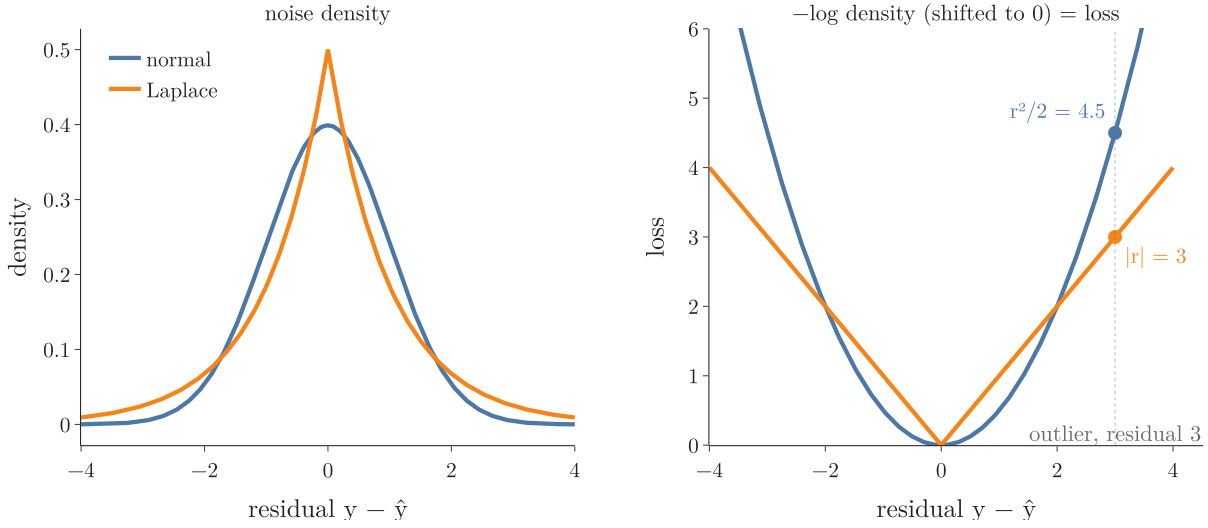


Figure 3: Left: normal noise and Laplace noise. Right: minus the log of each density, shifted to start at 0: the normal gives the parabola $r^2/2$ (squared error), the Laplace gives the V $|r|$ (absolute error). At a residual of 3 they charge 4.5 and 3

In Figure 3, watch the outlier line at a residual of 3: the parabola charges more than the V, and the gap keeps growing for points further out.

4. A Bernoulli target gives the log loss

Key point: With a 0/1 target and predicted probability $\hat{y}$, the Bernoulli PMF $\hat{y}^y(1-\hat{y})^{1-y}$ has minus log $-[y\log\hat{y} + (1-y)\log(1-\hat{y})]$: the log loss term for one observation.

4.1 Why least squares cannot fit logistic regression

Key point: On the log-odds axis every observation sits at $+\infty$ or $-\infty$, so every **residual** (G-705) is infinite. Least squares has nothing to minimise; maximum likelihood does.

We use 8 Iris flowers (the **Iris dataset**, G-973): 4 versicolor and 4 virginica, with petal widths 1.0, 1.3, 1.5, 1.7 cm (versicolor) and 1.5, 1.8, 2.0, 2.3 cm (virginica). The target is 1 for virginica and 0 for versicolor. Logistic regression draws a straight line, not on the probability axis, but on the **log-odds** (G-1116) axis, $\log\frac{p}{1-p}$; the sigmoid turns it back into a probability (see the sigmoid function).

The log-odds of a probability of 1 is plus infinity, and of a probability of 0 it is minus infinity:

$$\log(1/0) = +\infty$$

So on the log-odds axis the virginica flowers sit at $+\infty$ and the versicolor flowers at $-\infty$ (Figure 4, the panel "log-odds: flowers at $\pm\infty$", triangles). The residual from any straight line to a point at infinity is infinite, so the sum of squared residuals is infinite for every line, and least squares cannot compare two lines.

4.2 Project, convert, multiply, turn

> **Key point:** Each flower gets its probability from the line; its likelihood is p if it is virginica and $1 - p$ if it is versicolor. The best line has the largest product.

Maximum likelihood scores a candidate line in four steps:

1. **Project:** move each flower straight onto the line. Its height there is its candidate **log-odds** (G-1116).
2. **Convert:** the sigmoid turns each log-odds z into a probability of virginica:

   $$p = \frac{1}{1 + e^{-z}}$$

   On the probability axis the straight line becomes an S-shaped curve (Figure 4, the panel "probability: bar = likelihood").
3. **Multiply:** a virginica flower's likelihood is its p; a versicolor flower's likelihood is $1 - p$, the probability of not being virginica. The bars in Figure 4 (the panel "probability: bar = likelihood") have exactly these lengths. In logs, the log-likelihood is the sum of the logs of the eight bars.
4. **Turn:** change the slope of the line and score it again. Keep the line with the largest log-likelihood.

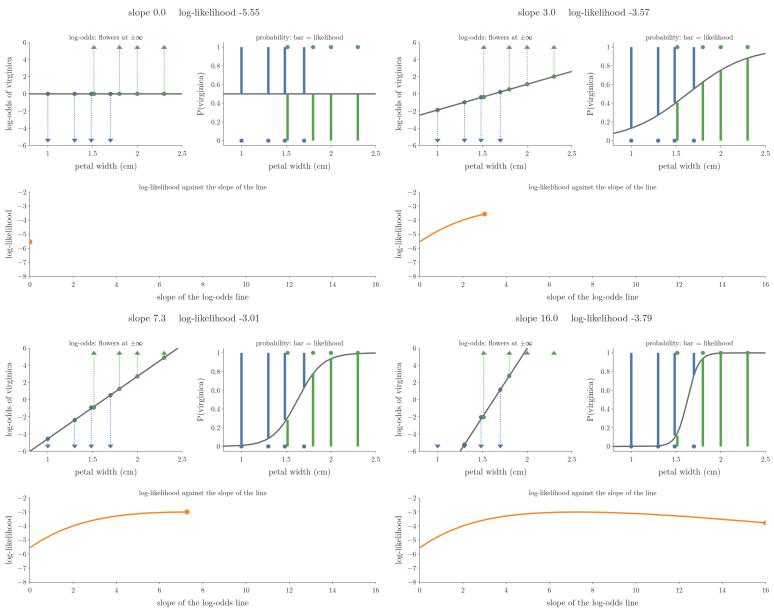


Figure 4: Logistic regression on 8 Iris flowers (green: virginica, blue: versicolor). The log-odds line turns about the point where it crosses 0. Panel "log-odds: flowers at $\pm\infty$": on the log-odds axis the flowers sit at plus or minus infinity (triangles); dots are their projections onto the line. Panel "probability: bar = likelihood": the same line as a probability curve; each bar is a flower's likelihood. Panel "log-likelihood against the slope of the line": the log-likelihood against the slope, highest at slope 7.27. Idea after StatQuest, "Logistic Regression Details Pt 2: Maximum Likelihood"

In Figure 4, watch the panel "log-likelihood against the slope of the line". The flat line (slope 0) gives every flower $p = 0.5$ and log-likelihood $8 \log 0.5 = -5.55$. Steeper lines lengthen most bars and the log-likelihood rises to -3.01 at slope 7.27. Steeper still, the versicolor flower at 1.7 cm and the virginica flower at 1.5 cm, which sit on the wrong sides, get very short bars, and the log-likelihood falls again (-3.79 at slope 16). An optimiser does the same search, but each turn goes in the direction that raises the log-likelihood (Notebook, Section 2).

> **Another way to see it:** Comparing two models compared just two fixed decision boundaries on four points. For each point it took the probability the model gave to the point's **true** class, multiplied the four numbers, and called the model with the larger product better: 0.090 for model 1 against 0.176 for model 2. The two views do the same thing. Figure 4 tries a whole family of lines instead of two, and its bars are exactly "the probability of the true class".

4.3 The Bernoulli model behind the log loss

Maximum likelihood in the log loss built the binary cross entropy by multiplying the probabilities of the true classes and taking minus the log. It found the formula $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ by noticing that the y factors switch the right term on. The NLL view shows where that formula comes from.

1. **In words:** model the target as a Bernoulli trial (see the PMF as one formula) whose success probability is the sigmoid output $\hat{y} = \sigma(w \cdot x)$. Minus the log of its PMF is the log loss term.

2. **Formula:**

$$p(y \mid x, \theta) = \hat{y}^{\,y}(1 - \hat{y})^{1-y}$$

   Taking minus the log:

$$-\log p(y \mid x, \theta) = -\Big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\Big]$$

   The log brings the exponents y and $1 - y$ down as multipliers. Summed over the observations, this is the binary cross entropy; divided by n, the log loss.

3. **Example:** the four observations of comparing two models, model 1: targets 1, 0, 1, 0 with $\hat{y} = 0.7, 0.6, 0.4, 0.2$. The Bernoulli PMF gives the probability of each true target:

$$0.7^1 \times 0.3^0 = 0.7$$

$$0.6^0 \times 0.4^1 = 0.4$$

   The third and fourth are 0.4 and 0.8. The NLL is minus the log of their product:

$$-\log(0.7 \times 0.4 \times 0.4 \times 0.8) = 2.41$$

   This is the cross entropy found there.

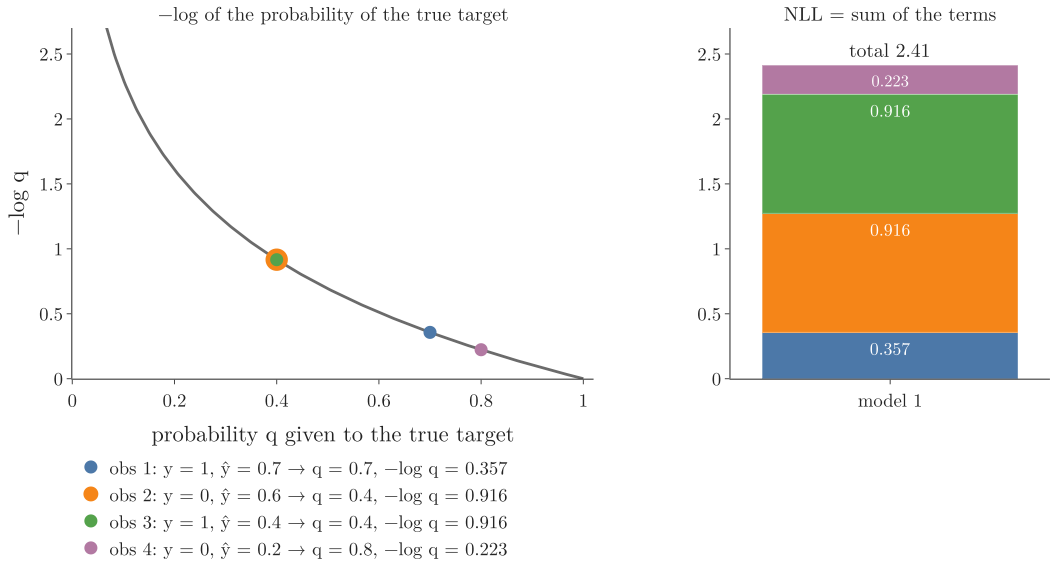


Figure 5: Model 1 observation by observation. Left: each observation's probability q of its true target on the curve −log q. Right: the four terms stacked into the NLL, 2.41

In Figure 5, watch where each point sits on the curve: observations 2 and 3, whose true target got only 0.4, contribute most of the 2.41.

So logistic regression is **maximum likelihood estimation** (G-1191) for a Bernoulli model whose probability comes from a sigmoid. Unlike Gaussian noise, the weights sit inside the sigmoid and the logs, so there is no closed form and we use gradient descent.

> **Extra:** The log loss of logistic regression has a single lowest point. Bishop (§4.3.3) shows that its Hessian is positive definite, so the loss is a bowl with a unique minimum, and gradient descent cannot get stuck on a worse one. The log-likelihood panel of Figure 4 has one peak for the same reason. Several minima appear only when a neural network sits between the weights and the loss (a tiny neural network: two minima).

5. A categorical target gives the cross entropy

> **Key point:** With several classes, the model gives each class a probability; the loss is minus the log of the probability it gave to the true class, and nothing else. That is the categorical cross entropy.

In plain words. Suppose a flower can be one of three species. The model gives each species a probability, and the three add up to 1. If the flower really is species 1, we read off the probability the model gave to species 1 and ignore the others. The higher it is, the smaller the loss. A softmax output provides exactly such probabilities (see the softmax function). The distribution that gives probabilities to K outcomes is the **categorical distribution** (G-352), the Bernoulli distribution extended from 2 outcomes to K.

Worked example. Three classes, softmax output $(0.7, 0.2, 0.1)$. The true class is the first. Write the true class as a **one-hot** list: 1 for the true class, 0 for the others, so $\mathbf{y} = (1, 0, 0)$. The model's probabilities are $\hat{y} = (\hat{y}_1, \hat{y}_2, \hat{y}_3) = (0.7, 0.2, 0.1)$.

Class k	y_k	$\hat{y}_k$	Factor $\hat{y}_k^{y_k}$	Term $y_k \log \hat{y}_k$
1	1	0.7	$0.7^1 = 0.7$	$1 \times \log 0.7 = -0.357$
2	0	0.2	$0.2^0 = 1$	$0 \times \log 0.2 = 0$
3	0	0.1	$0.1^0 = 1$	$0 \times \log 0.1 = 0$

The product of the factors is the probability of the true class:

$$0.7 \times 1 \times 1 = 0.7$$

The sum of the terms is -0.357. The loss is minus that:

$$-\log 0.7 = 0.357$$

The formal version. For K classes, $\prod_{k=1}^{K}$ means "multiply the factors for $k = 1$ to K" and $\sum_{k=1}^{K}$ means "add the terms". Every factor with $y_k = 0$ equals 1, so only the true class remains:

$$p(\mathbf{y} \mid x, \theta) = \prod_{k=1}^{K} \hat{y}_k^{y_k}$$

Taking minus the log:

$$-\log p(\mathbf{y} \mid x, \theta) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

Check: $K = 3$ gives, as in the table:

$$-(1 \times \log 0.7 + 0 + 0) = 0.357$$

Minus its log is the **categorical cross entropy**.

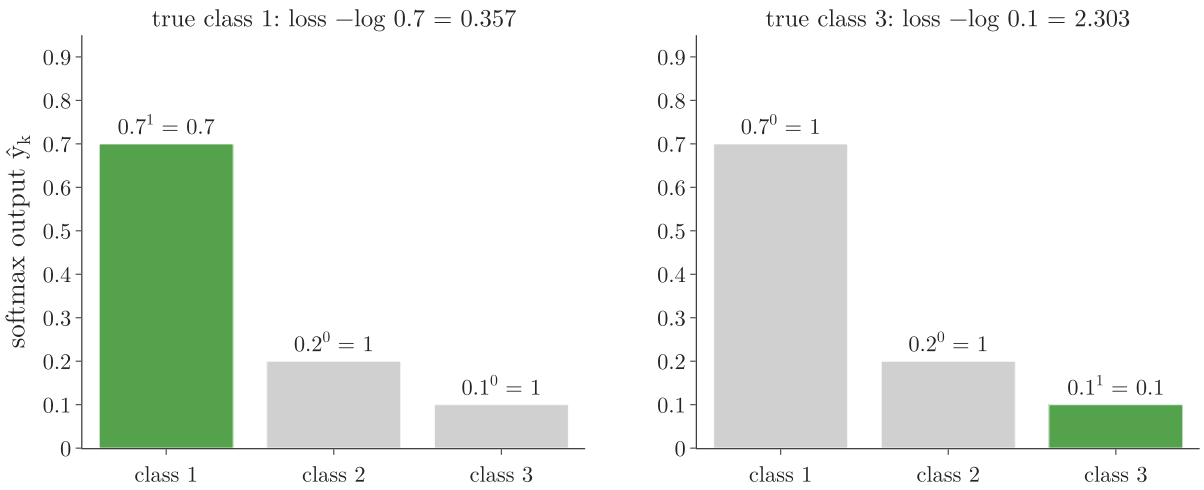


Figure 6: The softmax output (0.7, 0.2, 0.1). Each bar is labelled with its factor $\hat{y}$ to the power y; only the true class (green) keeps its probability, every other factor is 1. True class 1: loss 0.357; true class 3: loss 2.303

Figure 6 draws these bars. Watch the green bar: the loss only reads the probability of the true class, so the same output costs 0.357 when the model is right and 2.303 when the true class is the one it rated 0.1.

The categorical cross entropy is the loss of softmax regression and of every classification network (categorical cross-entropy). Training a classifier by minimising cross entropy is **maximum likelihood estimation** (G-1191).

With several observations, the losses add up. A small network that classifies Iris flowers gives three training flowers these softmax probabilities for their true species: 0.57 (a setosa), 0.58 (a virginica) and 0.52 (a versicolor). Their losses, one per line:

$$-\ln 0.57 = 0.56$$

$$-\ln 0.58 = 0.54$$

$$-\ln 0.52 = 0.65$$

The total is the number that backpropagation then tries to lower:

$$0.56 + 0.54 + 0.65 = 1.76$$

5.1 Why not squared residuals?

> **Key point:** For a probability, the squared **residual** (G-705) $(1-p)^2$ never exceeds 1 and stays almost flat for bad predictions. Cross entropy $-\ln p$ grows without limit and is steep exactly where the prediction is bad, so bad predictions get large corrections.

The observed probability of the true class is 1, so each flower also has a residual, $1-p$. For the setosa above:

$$1 - 0.57 = 0.43$$

We could square the residuals and add them, as for regression. Figure 7 shows why we do not.

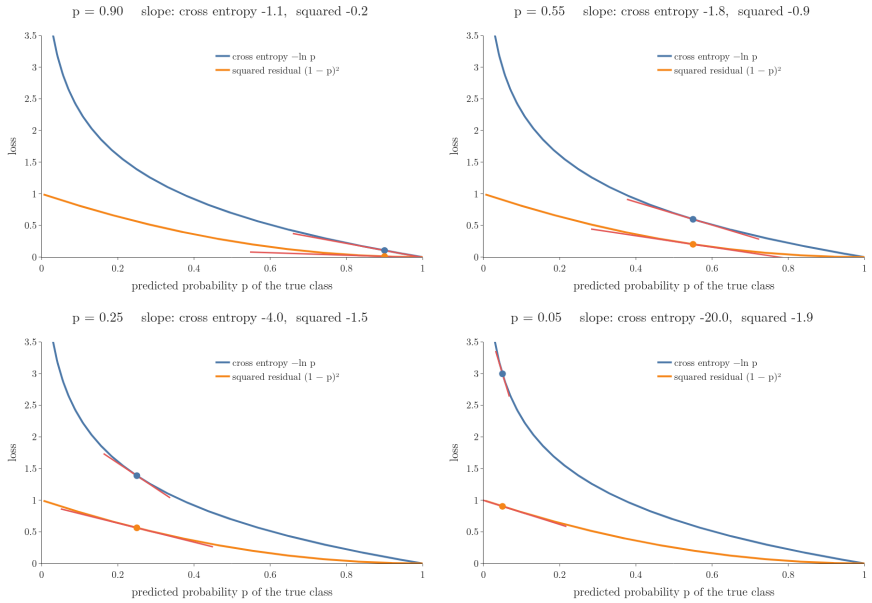


Figure 7: The predicted probability p of the true class falls from 0.95 to 0.05. Blue: cross entropy $-\ln$ p. Orange: the squared residual $(1 - \text{p})^2$. Red: the tangent lines at the current p. At p = 0.05 the slopes are -20 and -1.9. Idea after StatQuest, "Neural Networks Part 6: Cross Entropy"

1. **In words:** as the prediction gets worse, cross entropy explodes, while the squared residual creeps up to at most 1.
2. **Mechanism:** a gradient step is proportional to the slope of the loss (how far to move). A steeper slope means a larger step towards a better prediction.
3. **Example:** at $p = 0.05$, a very bad prediction. The slope of each loss, one line each:

   Cross entropy:

$$\text{slope} = -\frac{1}{p} = -\frac{1}{0.05} = -20$$

Squared residual $(1-p)^2$:

$$\text{slope} = -2(1-p)$$

$$\text{slope} = -2 \times 0.95 = -1.9$$

The cross entropy slope is about ten times steeper.

In Figure 7, watch the two red tangent lines as the dot slides to the left: the blue one turns almost vertical, the orange one barely tilts.

Sections 3 to 5 in one table: each model of the target, its NLL for one observation, and the loss that NLL is.

Model of the target	NLL per observation	Name of the loss
$N(\hat{y}, \sigma^2)$, any real y	$(y-\hat{y})^2/(2\sigma^2) + \text{const}$	squared error (MSE)
$\text{Laplace}(\hat{y}, b)$	$\lvert y-\hat{y}\rvert/b + \text{const}$	absolute error (MAE)
$\text{Bern}(\hat{y})$, $y \in \{0,1\}$	$-[y \log \hat{y} + (1-y)\log(1-\hat{y})]$	log loss
$\text{Cat}(\hat{y}_1, \ldots, \hat{y}_K)$, one-hot $\mathbf{y}$	$-\sum_k y_k \log \hat{y}_k$	categorical cross entropy

6. Maximum likelihood overfits

> **Key point:** Maximum likelihood uses all the freedom a model has to match the training data, **noise** (G-1326) included. With many parameters and few observations, the training NLL keeps falling while the error on new data explodes.

A model that is too flexible learns the noise of its training data and fails on new data: this is **overfitting** (see **overfitting** (G-1429) and bias-variance). Maximum likelihood has no brake against overfitting. The method is told to make the observed data as likely as possible, and a model that passes through every point does that best.

Figure 8 repeats the experiment of MML §9.2.2, with our own random draw of the data (Notebook, Section 4). Ten points come from this function plus noise with $\sigma = 0.2$:

$$y = -\sin(x/5) + \cos(x)$$

We fit polynomials of degree 0 to 9 by maximum likelihood with **Gaussian noise** (G-831), which is least squares on polynomial features (see adding powers as new features).

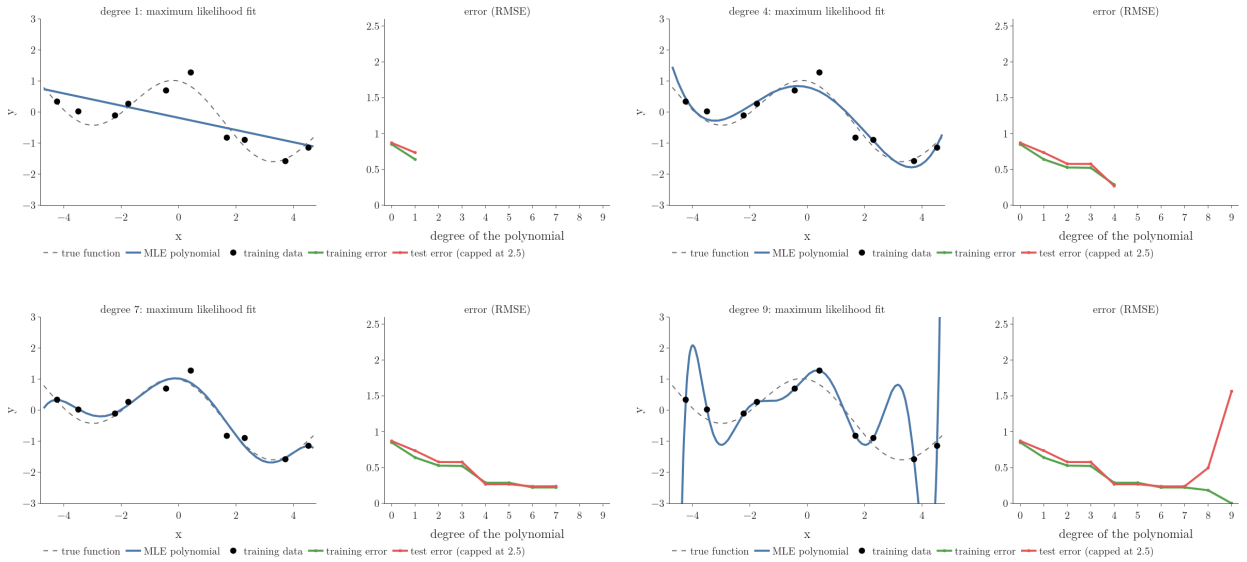


Figure 8: Maximum likelihood polynomials of degree 0 to 9 on ten points. Right: the training error falls to 0 while the test error, after a minimum, shoots up

Degree	0	1	2	4	6	8	9
Training RMSE	0.85	0.64	0.53	0.29	0.22	0.18	0.00
Test RMSE (200 new points)	0.87	0.74	0.58	0.27	0.24	0.49	1.56

- The **training error never rises** as the degree grows (the book observes the same). The reason is a one-line argument: every polynomial of degree $M-1$ is also a polynomial of degree M with last coefficient 0, so the best degree M fit can do no worse on the training data.
- At degree 9 there are 10 coefficients for 10 points. The polynomial passes through every point, the training error is 0, and the curve swings wildly between the points.
- The **test error** is lowest around degree 6 and then rises fast.

With more parameters than observations the MLE is not even unique (MML §9.2.2): the matrix $X^\top X$ in the normal equation cannot be inverted, and infinitely many coefficient vectors fit the data equally well. The next section adds the missing brake.

7. MAP estimation: maximum likelihood plus a prior

> **Key point:** MAP estimation maximises likelihood × **prior** (G-1564). Its negative log is the NLL plus a penalty on the parameters: a Gaussian prior gives the ridge penalty, a Laplace prior the lasso penalty.

7.1 The posterior over the parameters

> **Key point:** Before seeing data we already believe the weights are probably small. MAP picks the weights that fit the data and also agree with that belief.

In plain words. A student fits a line to four points. The data alone says "slope 2.01". But a sensible person says: "slopes much bigger than 2 are rare in my problem, so I will accept a slightly worse fit for a more ordinary slope". That prior belief is written as a distribution over the weights. The data score and the belief score are then combined, and the best combination wins.

Worked example on the four points of Section 3 ($x = 1, 2, 3, 4$; $y = 1.8, 4.3, 5.7, 8.2$; line $\hat{y} = wx$; **noise** (G-1326) $\sigma = 1$). The belief: the slope w is normal around 0 with spread 0.5. The cost of a slope is the NLL part plus the prior part (constants dropped):

Slope w	Squared errors	NLL part $\frac{1}{2}\sum(y - wx)^2$	Prior part $\frac{w^2}{2 \times 0.5^2} = 2w^2$	Total
2.01 (the MLE)	0.257	0.13	$2 \times 2.01^2 = 8.08$	8.21
1.77	1.985	0.99	$2 \times 1.77^2 = 6.27$	7.26

The data alone prefers 2.01. With the belief, 1.77 has the smaller total and wins: that is the MAP slope (Section 7.2 derives it).

The formal version. MML (§9.2.3) observes that parameter values often become large when a model overfits, and proposes to state beforehand which values are plausible. A **prior** (G-1564) $p(\theta)$ is that belief as a distribution over the parameters. Bayes' theorem (see the names of the four parts) combines it with the likelihood $p(\text{data} \mid \theta)$ into a **posterior** (G-1535), the belief after seeing the data:

$$p(\theta \mid \text{data}) = \frac{p(\text{data} \mid \theta)\, p(\theta)}{p(\text{data})}$$

The evidence $p(\text{data})$ does not depend on θ, so for finding the best θ we can drop it, as Naive Bayes dropped it when comparing classes (the strategy: compare posteriors). The **maximum a posteriori** (**MAP**) estimate (G-1189) is the θ with the largest posterior. In logs, it minimises the NLL plus minus the log of the prior:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta}\, p(\text{data} \mid \theta)\, p(\theta)$$

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \Big[\text{NLL}(\theta) - \log p(\theta)\Big]$$

Check: the bracket is the "Total" column of the table. With a flat prior, the same for every θ, $-\log p(\theta)$ is a constant and MAP gives the MLE. The prior only matters when it prefers some values over others.

The formula and the MAP rule used the MAP rule to pick a class. Here the same principle picks parameter values.

7.2 A Gaussian prior gives ridge regression

> **Key point:** With **prior** (G-1564) $\theta_j \sim N(0, b^2)$ and **Gaussian noise** (G-831) σ, MAP minimises $\sum(y_i - \hat{y}_i)^2 + \lambda \sum_j \theta_j^2$ with $\lambda = \sigma^2/b^2$.

First, the penalty itself, from the idea: penalise large coefficients. With only two training points, the least squares line passes through both, with zero error, and its slope can be far too steep for new data. **Ridge regression** (G-1691) adds λ times the squared slope to the squared errors. With $\lambda = 1$ on the two points $(1, 2)$ and $(3, 5)$, the least squares line has slope 1.5 and costs:

$$0 + 1.5^2 = 2.25$$

A flatter line with slope 0.9 costs less, so it wins:

$$0.72 + 0.9^2 = 1.53$$

The penalty looks like an arbitrary fix. MAP shows where it comes from:

1. **In words:** a normal prior centred on 0 says each weight is probably small. Minus its log is the squared weight over $2b^2$, plus a constant. Added to the Gaussian NLL and multiplied by $2\sigma^2$, this is the ridge loss of the ridge solution for many features.
2. **Formula** (MML equations 9.28 and 9.33):

$$\text{NLL}(\theta) - \log p(\theta)$$

$$= \frac{1}{2\sigma^2} \sum_i (y_i - \hat{y}_i)^2 + \frac{1}{2b^2} \sum_j \theta_j^2 + \text{const}$$

$$\propto \sum_i (y_i - \hat{y}_i)^2 + \frac{\sigma^2}{b^2} \sum_j \theta_j^2$$

So $\lambda = \sigma^2/b^2$: a narrow prior (small b) or noisy data (large σ) means strong regularisation.

3. **Example:** the four points of Section 3 with $\sigma = 1$ and prior $w \sim N(0, 0.5^2)$, so:

$$\lambda = 1/0.25 = 4$$

As in the ridge solution for one feature, λ is added to the bottom of the fraction:

$$\hat{w}_{\text{MAP}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda}$$

$$\hat{w}_{\text{MAP}} = \frac{60.3}{30 + 4} = 1.77$$

The prior pulls the slope from the MLE 2.01 towards 0.

Figure 9 applies this to the degree-9 polynomial of Figure 8, with prior $N(0, 0.1^2)$ on every coefficient and $\sigma = 0.2$, so:

$$\lambda = 0.04/0.01 = 4$$

The MLE swings through every point; the MAP curve stays close to the true function, and its test RMSE drops from 1.56 to 0.54.

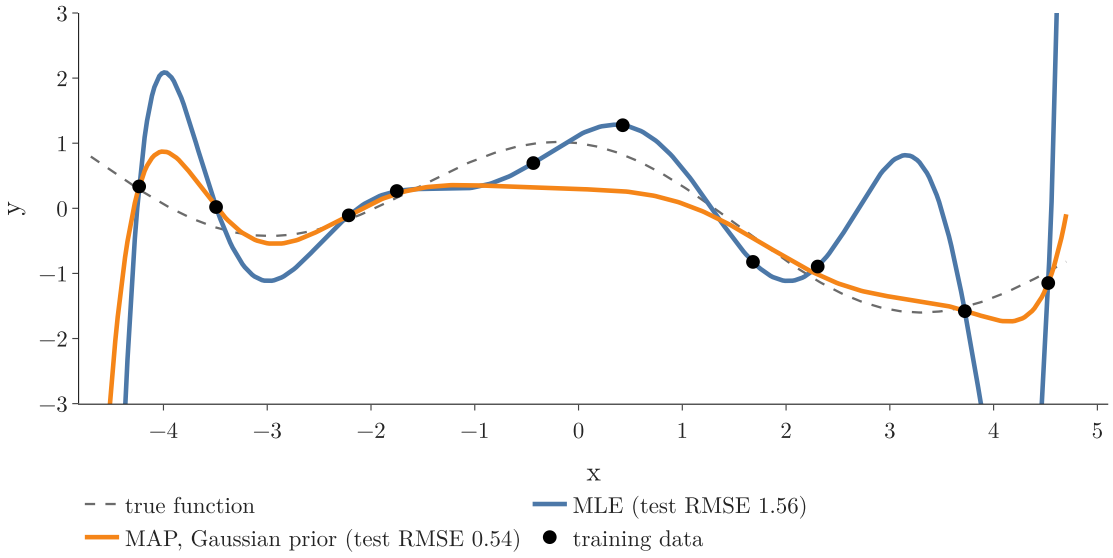


Figure 9: Degree-9 polynomial on ten points: maximum likelihood versus MAP with a Gaussian prior

7.3 A Laplace prior gives the lasso

> **Key point:** With a Laplace **prior** (G-1564), minus the log prior is $\sum_j |\theta_j|/b$: the L1 penalty of the lasso.

Compare the two penalties on a weight of 0.3 and a tiny weight of 0.03, both with $\lambda = 4$:

Weight	L2 penalty $4w^2$	L1 penalty $4\|w\|$
0.3	$4 \times 0.09 = 0.36$	$4 \times 0.3 = 1.2$
0.03	$4 \times 0.0009 = 0.0036$	$4 \times 0.03 = 0.12$

The squared penalty almost vanishes for a tiny weight, so it stops pushing; the absolute penalty keeps pushing at the same rate down to 0. A prior with this behaviour, a sharp peak at 0, is the Laplace prior.

1. **In words:** the Laplace prior has a sharp peak at 0, so it believes many weights are exactly 0. Minus its log is the absolute value of each weight divided by b.
2. **Formula:** the prior of each weight is:

   $$p(\theta_j) = e^{-|\theta_j|/b}/(2b)$$

   Then:

   $$\text{NLL}(\theta) - \log p(\theta)$$

   $$\propto \sum_i (y_i - \hat{y}_i)^2 + \frac{2\sigma^2}{b} \sum_j |\theta_j|$$

   The result is the loss of **lasso regression** (G-1047), with $\lambda = 2\sigma^2/b$.
3. **Example:** with $\sigma = 1$ and $b = 0.5$:

   $$\lambda = 2/0.5 = 4$$

   A weight of 0.3 costs this penalty:

   $$4 \times 0.3 = 1.2$$

MML (§9.5) states this equivalence of the Laplace prior and the lasso. The term $|\theta_j|/b$ has a corner at 0, the same corner of $|m|$ that lets lasso coefficients reach exactly 0 (why lasso reaches 0).

Extra: MAP still returns a single **point estimate** (G-1507), one set of parameter values (MML §9.2.4). The book notes (Section 9.2.3, Example 9.6) that a prior can push back **overfitting** (G-1429) but is not a general cure, and continues in Section 9.3 with Bayesian linear regression, which keeps the whole posterior distribution and averages the predictions of all plausible parameter values.

8. Summary

Ingredient	Becomes	Example in this Note	Why it matters
Gaussian noise + MLE	least squares, MSE	slope $\hat{w} = 60.3/30 = 2.01$	least squares, MSE and maximum likelihood pick the same line
MLE of the noise variance	mean squared residual	$\hat{\sigma}^2 = 0.064$	the training MSE says how far points scatter around the line
Laplace noise + MLE	MAE (L1 loss)		heavier tails, so the loss is more robust to outliers
Bernoulli target + MLE	log loss	NLL 2.41 for model 1	least squares cannot fit logistic regression, because every residual on the log-odds axis is infinite
Categorical target + MLE	categorical cross entropy	$-\log 0.7 = 0.357$	steep where the prediction is bad, so bad predictions get large corrections
Gaussian prior + MAP	ridge, $\lambda = \sigma^2/b^2$	$\hat{w} = 60.3/34 = 1.77$	the ridge penalty is a belief that weights are small, not an arbitrary fix
Laplace prior + MAP	lasso, $\lambda = 2\sigma^2/b$		the sharp peak at 0 is why lasso weights can reach exactly 0

- A **probabilistic model** (G-1567) gives a distribution $p(y \mid x, \theta)$ over the target; training minimises its NLL, so the model learns to give high probability to the targets that really happened.
- The choice of distribution for the target decides the loss: normal gives squared error, Bernoulli gives log loss, categorical gives cross entropy. So each familiar loss follows from an assumption about the target.
- Maximum likelihood overfits flexible models: the training error never rises with more parameters, because maximum likelihood uses all the model's freedom to match the training data, **noise** (G-1326) included.
- MAP adds $-\log p(\theta)$ to the NLL; a Gaussian prior is ridge, a Laplace prior is lasso. So the prior is the brake against **overfitting** (G-1429): on the degree-9 polynomial it cut the test RMSE from 1.56 to 0.54.

9. Sources

Built from

- Starmer, J. (StatQuest), "The Main Ideas of Fitting a Line to Data (The Main Ideas of Least Squares and Linear Regression)", YouTube, https://www.youtube.com/watch?v=PaFPbb66DxQ The least squares recap that opens Section 3.
- Starmer, J. (StatQuest), "Logistic Regression Details Pt 2: Maximum Likelihood", YouTube, https://www.youtube.com/watch?v=BfKanl1aSG0. Sections 4.1 and 4.2 (Figure 4).
- CampusX, "Logistic Regression Part 4 | Loss Function | Maximum Likelihood | Binary Cross Entropy", YouTube, https://www.youtube.com/watch?v=6bXOo0sxY5c. The two-model comparison in Section 4.2 (through the comparison of two models).
- Starmer, J. (StatQuest), "Neural Networks Part 6: Cross Entropy", YouTube, https://www.youtube.com/watch?v=6ArSys5qHAU. The Iris example of Section 5 and Section 5.1 (Figure 7).
- CampusX, "Loss Functions in Deep Learning | Deep Learning | CampusX", YouTube, https://www.youtube.com/watch?v=gb5nm_3jBIo. Section 5: only the true class's term survives a one-hot target.
- Starmer, J. (StatQuest), "Regularization Part 1: Ridge (L2) Regression", YouTube, https://www.youtube.com/watch?v=Q81RR3yKn30. The penalty recap that opens Section 7.2.
- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Free PDF at mml-book.github.io. §8.3.1 (Gaussian likelihood gives least squares, eq. 8.18), §8.3.2 (MAP), §9.2.1 (MLE for linear regression, Hessian remark, noise variance eq. 9.22), §9.2.2 (overfitting experiment), §9.2.3–9.2.4 (MAP as regularisation, eqs. 9.28 and 9.33), §9.5 (Laplace prior and lasso).

Other references

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. §4.3.3 (the Hessian of the logistic regression error is positive definite, so the error has a unique minimum).
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. §7.4 (robust linear regression with the Laplace likelihood).
- scikit-learn documentation: `LogisticRegression` (C = inf for no penalty) and `Ridge`, used in the Notebook checks.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Prior $p(\theta)$ (G-1564)	A distribution over the parameters expressing what we believe before seeing data.
Probabilistic model (G-1567)	A model that gives a probability for every possible target value, $p(y \mid x, \theta)$, instead of a single predicted value.
Noise (ε) (G-1326)	The random part of a target that the model's prediction does not explain.
Gaussian noise (G-831)	Random errors that follow a bell curve centred on 0, $N(0, \sigma^2)$. Assuming them and maximising the likelihood (MLE) gives the squared-error loss.
Laplace distribution (G-1044)	A peaked, heavy-tailed distribution with density $e^{-\lvert x-\mu\rvert/b}/(2b)$.
Posterior $p(\theta \mid \text{data})$ (G-1535)	The distribution over the parameters after seeing the data; proportional to likelihood $\times$ prior.
Maximum a posteriori (MAP) estimation (G-1189)	Choosing the parameters with the largest posterior probability given the data, which weighs the likelihood by a prior belief about the parameters; in practice, minimise the negative log-likelihood minus the log prior.
Maximum likelihood estimation (MLE) (G-1191)	Fitting parameters by making the likelihood of the observed data as large as possible.
Error (residual) (G-705)	The gap between an actual value and the model's prediction on one data point: actual minus predicted.
Residual sum of squares (sum of squared errors, SSE) (G-1684)	The squares of all the errors (residuals) added up; the quantity the best-fit line makes smallest, and a regression tree splits where the SSE of the two sides is smallest.
Ordinary least squares (OLS, least squares) (G-1406)	The way linear regression finds its line with one formula (a closed-form method): the line with the smallest sum of squared errors.
Iris dataset (G-973)	150 iris flowers of three species, with four measurements each; a classic classification dataset.
Log-odds (G-1116)	The natural log of the odds, $\ln(p/(1-p))$: it turns a probability between 0 and 1 into any number (0 at $p = 0.5$), so a model such as logistic regression can predict it with a straight line, and the sigmoid turns it back into a probability.

Term	Meaning
Categorical distribution (G-352)	The distribution of one trial with more than two outcomes; Bernoulli is its two-outcome case.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Ridge regression (G-1691)	Linear regression with a penalty on the sum of squared coefficients added to the loss; the penalty keeps the coefficients small, which reduces overfitting (L2 regularisation).
Lasso regression (G-1047)	Linear regression with a penalty on the sum of absolute coefficients (L1); it shrinks coefficients and can set some exactly to 0, which removes those features.
Point estimate (G-1507)	A single number computed from sample data as the best guess for an unknown population parameter.